

取り組みたい研究テーマ: Evaluating Host-Level Context for Detecting Short-Lived DNS-over-HTTPS Traffic

1. これまでの修学内容

My undergraduate thesis at the University of Science and Technology of China studied robust TLS encrypted traffic classification under network perturbation. The starting point was a weakness of packet-length-sequence-based classifiers: although TLS hides payload contents, packet lengths remain observable, yet they are affected by transport behavior. Under congested network conditions, TCP-related dynamics such as retransmission, sequence shift, and packet merging can change the observed sequence. A classifier that relies on fixed packet positions may therefore learn transport artifacts rather than stable application-level patterns.

To address this issue, I designed TCP-Aware Contrastive Sequence (TAC-Seq), a contrastive representation learning framework for perturbed packet-length sequences, as shown in Figure 1. TAC-Seq was inspired by Rosetta's TCP-aware traffic augmentation [1], but I redesigned the learning framework: instead of reproducing Rosetta's BYOL-based approach, I used a SimCLR-style objective with NT-Xent loss [2]. The augmentation pool modeled subsequence repetition, subsequence shift, and Nagle-like packet-size variation. In this design, disturbed observations of the same original flow are used as related views, encouraging the model to learn features that remain meaningful under transmission-level variation.

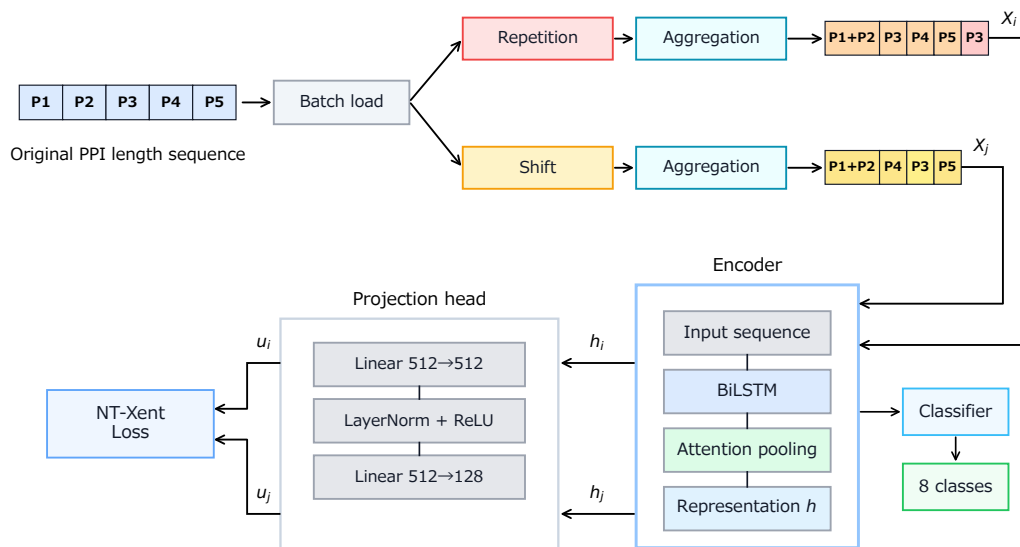


Figure 1. TAC-Seq framework implemented in my undergraduate thesis.

The model used self-supervised pretraining followed by downstream classification. For sequence representation, TAC-Seq used a BiLSTM-Attention encoder on packet-length sequences. During pretraining, two augmented views from the same original flow formed a positive pair, while views from other flows served as negative samples. The encoder output was passed through a projection head for contrastive learning; after pretraining, the projection head was removed and a classifier was trained on top of the frozen encoder representation. This separation allowed me to evaluate whether the encoder learned perturbation-robust sequence representations, rather than only optimizing a classifier for one fixed setting.

The evaluation used CESNET-TLS22 with an eight-class TLS classification task. To reflect an online classification setting, the input was limited to packet-length information from the first 30 packets of each flow. Robustness was tested under fixed FAST and RTO perturbation settings, and the method was compared with a linear packet-length baseline and a Rosetta-style implementation. I also disabled the packet-merging-related augmentation for ablation. The results showed that TAC-Seq outperformed the linear baseline in all settings and was stronger than the reproduced Rosetta result in most settings, especially under more severe RTO perturbations. Packet-size variation was useful in part of the evaluated settings, particularly for lighter FAST perturbations. At the same time, because the dataset did not provide timestamps, the Nagle-like component had to be treated as a heuristic approximation rather than a protocol-exact reconstruction. This limitation clarified that encrypted traffic analysis must align model design with observable protocol evidence.

2. 取り組みたい内容

2.1 Background and problem.

I would like to study practical security monitoring for encrypted traffic. My initial project focuses on DNS over HTTPS (DoH). DoH protects DNS queries by sending them over HTTPS [3]. This improves privacy, but it also makes unauthorized DNS communication harder for a network operator to identify because DoH can resemble ordinary web traffic.

Existing DoH detectors have made substantial progress. However, a recent comparative analysis reports that some approaches depend on known resolver addresses, traffic burstiness, or stable training conditions, while short or single-query DoH flows to private resolvers remain difficult to detect [4]. This matters because malware can use a private resolver and low-volume communication to reduce visibility.

The key difficulty is the unit of observation. A flow-level classifier sees only a few encrypted packets in a short DoH flow, so it may not have enough evidence unless it uses resolver identity or long-flow statistics. A monitored host, however, may show temporal evidence around that flow, such as repeated short HTTPS connections, short idle gaps, similar size-direction exchanges, or changes in low-volume flow density within a short window, rather than relying on one sparse flow sequence. This keeps the additional evidence tied to the same monitored host. This motivates my research question:

Research question: Can host-level temporal context improve the detection of short-lived DoH traffic to unseen private resolvers without inspecting payloads or relying on resolver addresses?

This question fits the Internet Architecture and Systems Laboratory’s practical direction. Katsura et al. showed that host-level aggregation can reduce IDS cost while maintaining high accuracy in IoT networks [5]. I will test this viewpoint when individual encrypted flows contain insufficient information.

2.2 Observation unit and feature design.

The proposed method changes the analysis unit from an isolated TLS flow to a host observation window. Each flow is represented by size, direction, inter-arrival-time, duration, and count metadata; each candidate flow is paired with a host window ending at the flow or at a specified detection delay.

The window statistics are derived from evidence missing in a short flow: short-flow counts, temporal spacing, burst and idle patterns, repeated size–direction profiles, and the share of low-volume encrypted communication. These interpretable groups will be ablated independently, so the study tests whether context supplies evidence rather than hiding resolver-specific shortcuts inside a larger model.

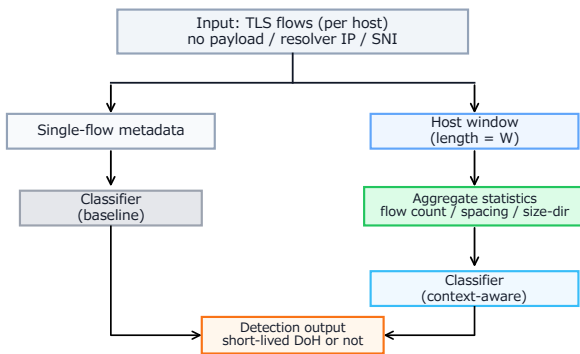


Figure 2. Flow-level and host-window observation units for short-lived DoH traffic.

2.3 Testbed and baselines.

I will construct a controlled testbed that captures ordinary HTTPS, benign DoH to public resolvers, and low-volume DoH to private resolvers, including short and single-query-like patterns. Collection will be repeated across separate sessions and network conditions. Labeled traces will be split by resolver and collection session, because a random flow-level split could place nearly identical traffic in both sets and produce an optimistic result.

Model inputs will be limited to metadata outside encryption. Payloads, resolver IP addresses, DNS names, SNI, ALPN, certificate subject names, and other semantic TLS handshake fields will be excluded; TLS handshake packets may contain only size, direction, and timing. The host identifier is used only to group flows into windows, not as a learned feature.

2.4 Evaluation plan.

The first comparison will use an address-list detector, a single-flow metadata classifier, and a lightweight host-level model using the window statistics above. The address-list result will be reported separately because it uses resolver identity and cannot recognize unseen private resolvers. Representation learning will be introduced only if simple aggregation fails to capture stable behavior.

Host-level aggregation has a trade-off. A longer observation window can provide more context, but it may also delay detection and combine unrelated HTTPS connections. I will therefore evaluate multiple window sizes and query frequencies. I will also add controlled delay and packet loss. The main metrics will be recall for short-lived DoH flows, false-positive rate on ordinary HTTPS traffic, F1 score, detection latency, memory use, and processing time.

RQ1	Does host-level context improve recall compared with a single-flow model for short-lived DoH traffic?
RQ2	Does the improvement remain when the private resolvers used for testing are absent from the training data?
RQ3	How do the observation window, network conditions, latency, and false-positive rate affect deployment practicality?

Table 1. Research questions and evaluation viewpoints.

RQ1 compares flow-level and host-level models under the same metadata restrictions. RQ2 uses disjoint resolver sets and collection sessions. RQ3 varies window size, query frequency, delay, and packet loss. Feature groups will be removed in turn to identify which context supplies evidence.

Variable	Controlled settings	Purpose
Resolver	public / private; seen / unseen	Avoid learning a resolver-specific shortcut.
Query form	single query / low rate / ordinary	Measure the difficult short-flow cases.
Network	baseline / added delay / packet loss	Test robustness under changing conditions.
Window	multiple durations	Quantify the recall–latency trade-off.

Table 2. Controlled variables in the proposed evaluation.

2.5 Research stages and feasibility.

The stages are: build a reproducible traffic-generation pipeline and reproduce a simple flow-level baseline; measure its limits and evaluate interpretable host-level statistics; and introduce representation learning only if simple aggregation does not capture stable behavior. This order keeps the project feasible and produces useful measurements even if the host-level model provides only a limited improvement.

2.6 Expected contribution and extension.

The expected contribution is a reproducible comparison of flow-level and host-level detection, including failure cases. The project does not assume that all DoH traffic is detectable from metadata; it will clarify when host context helps, which context features matter, and whether a lightweight detector is practical. Newer transports such as DNS over QUIC (DoQ) [6] will be reserved as later extensions.

2.7 Scope and validation criteria.

This proposal changes the unit of analysis from a single TLS flow to a host observation window, so it does not simply repeat my undergraduate study. The approach will be useful only if it improves short-lived DoH detection for unseen resolvers while keeping false positives, latency, and processing cost measurable. If improvement requires resolver identity, semantic handshake fields, or an impractically long window, I will report that limitation and preserve the settings, splits, and scripts needed to reproduce it.

References

- [1] R. Xie et al., “Rosetta: Enabling Robust TLS Encrypted Traffic Classification in Diverse Network Environments with TCP-Aware Traffic Augmentation,” *USENIX Security*, 2023.
- [2] T. Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations,” *ICML*, 2020.
- [3] P. Hoffman and P. McManus, “DNS Queries over HTTPS (DoH),” *IETF RFC 8484*, 2018.
- [4] M. Jerabek et al., “Comparative Analysis of DNS over HTTPS Detectors,” *Computer Networks*, 2024.
- [5] Y. Katsura et al., “Efficient IDS for IoT Networks Using Host-Based Data Aggregation and Multi-Entropy Analysis,” *IEEE Access*, 2025.
- [6] C. Huitema et al., “DNS over Dedicated QUIC Connections,” *IETF RFC 9250*, 2022.